Distributed Systems

Events

def An event is an occurrence at a single process that causes a transition in its local state. It is formally represented as:

$$e_i^k = \text{K-th event of process } i$$

Relationship "Happened Before"

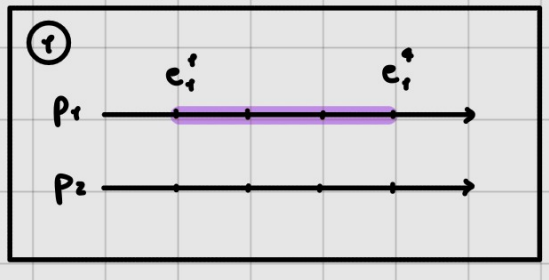
def The event e_i^l "happened before" e_j^m if:

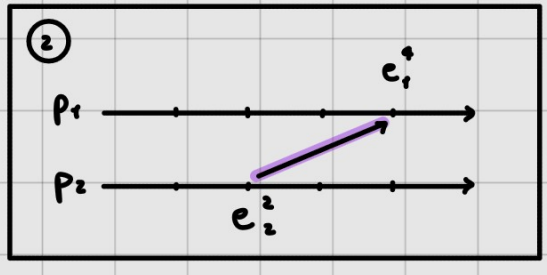
① $i = j$ and $l < m$ (same process and e^l happens before e^m)

OR

② e_i^l is the sender of the message received by e_j^m

NOTE: The relation "$\rightarrow$" is transitive $\left(e \rightarrow e' \wedge e' \rightarrow e'' \Rightarrow e \rightarrow e'' \right)$





Local State

def The Local State of a process is simply the "snapshot" of the process in a specific situation. It is formally represented as:

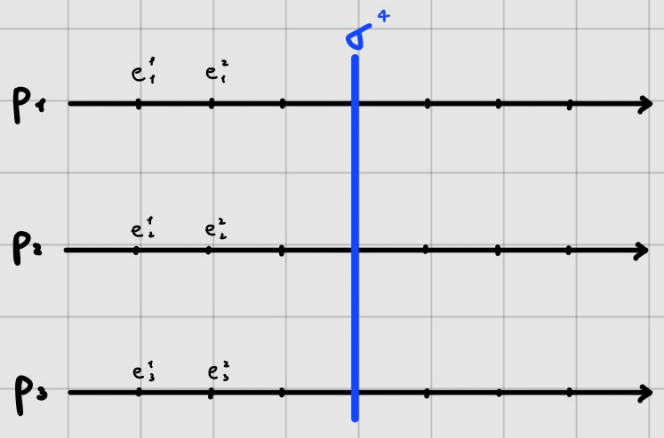
$$\sigma_i^k = \text{state of process } i \text{ after } k \text{ events } (e_i^1 \dots e_i^k)$$

Global State

def The Global State is a "snapshot" of the entire system. It is constructed from the union of the local state of every process. It is formally represented as:

$$\sigma^k = \text{state of all processes after } k \text{ events} = (\sigma_1^k, \sigma_2^k, \dots, \sigma_n^k)$$

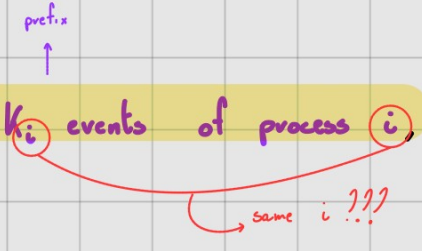
N.B. A global state is actually a "straight cut" on the time

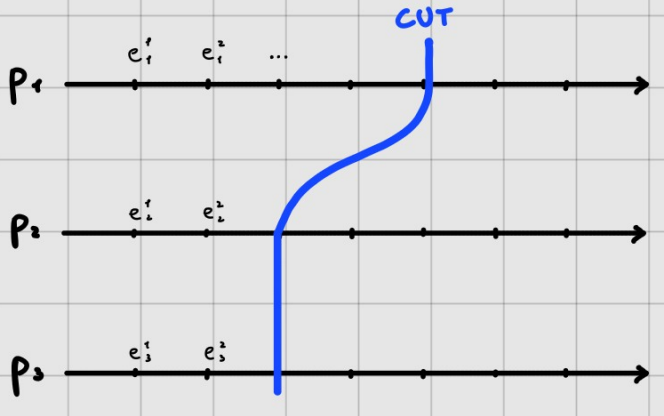


$$\sigma^4 = (\sigma_1^4, \sigma_2^4, \sigma_3^4)$$

Cut

def A "cut" is a set of events that contain the first k_i events of process i, for each process





$$C = (\sigma_1^5, \sigma_2^4, \sigma_3^4)$$

However, we are not interested in "every cut". We only need "consistent cuts".

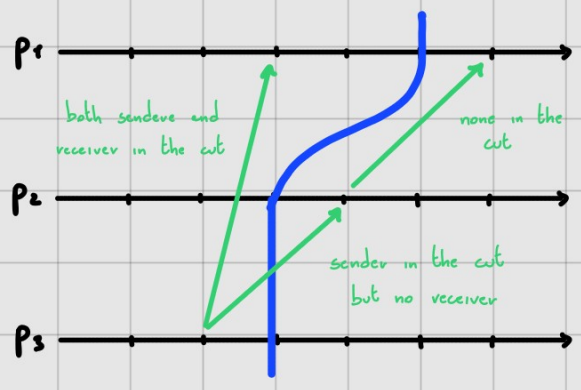
Consistent Cuts

def A cut C is consistent iff (if and only if): $e \longrightarrow e' \wedge e' \in C \Rightarrow e \in C$ (happens before)

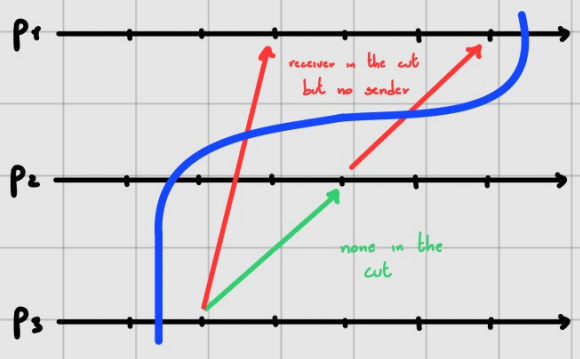
if e' receives a message from e, both e and e' must be in the cut

NON necessariamente il contrario. Se e manda un messaggio a e' ed $e \in C$, e' potrebbe anche NON appartenere a C

☑ CONSISTENT



☒ NOT CONSISTENT



Run

def A "run" is a sequence of events that represents a specific execution

As before, we can also define "what is a consistent run?"

def A run R is consistent iff (if and only if): $e \longrightarrow e' \Rightarrow e$ appears before e' in R (happens before)

And, basically, a Run is a sequence of Cuts. So, we could cut a Run to get a Cut.

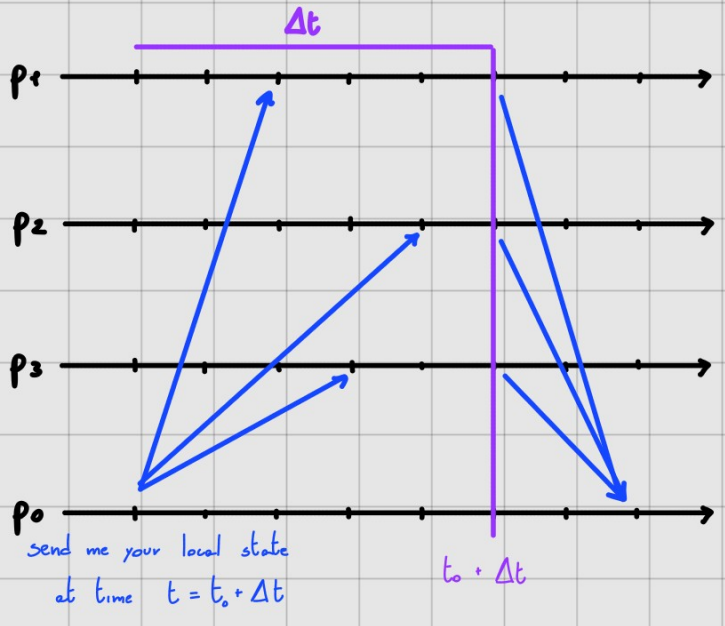
$$R = \overline{\overline{e_1^1 \; e_1^2 \; e_1^3}^{\,C'} \mid e_1^4 \; e_1^5} \mid e_1^6 \; \ldots \qquad (C'' \text{ and } C')$$

Lamport Clock

Let's start from an idealized world where capturing a consistent snapshot of the system's global state is very easy, thanks to 2 different assumptions:

- GLOBAL CLOCK: All processes share the exact same physical clock, meaning the clock drift is negligible or zero ($\Delta d = 0$)
- SYNCHRONOUS NETWORK: There is a know maximum Δt time needed to deliver messages.

Under these assumptions the following simple protocol works properly:

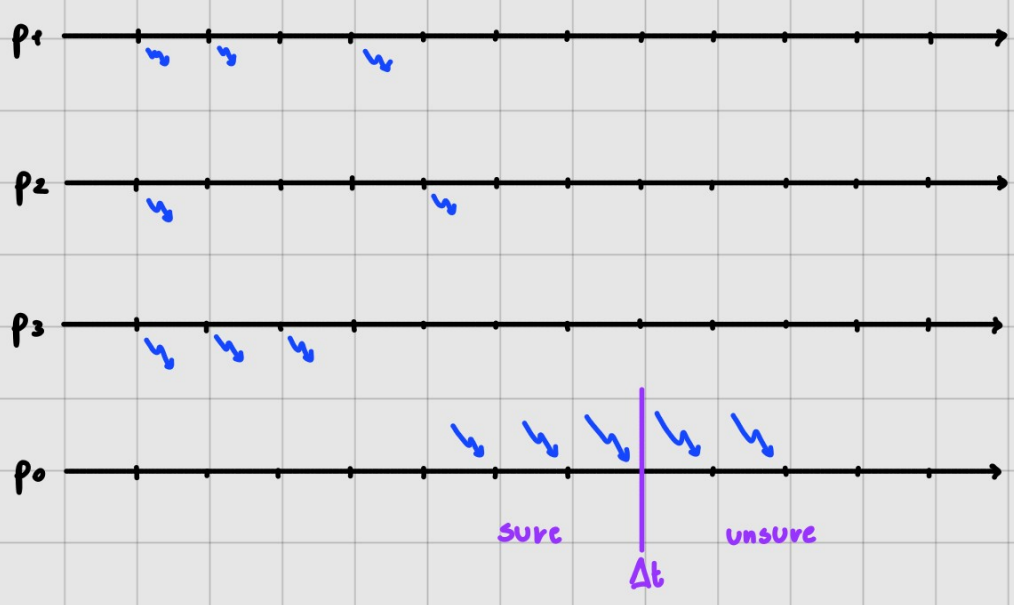


1) A coordinator process (p_0) broadcasts a request at time t_0, saying: "Send me your local state at tim $t = t_0 + \Delta t$"
2) Because the network is synchronous all processes receives the request in time (before Δt time)
3) Thank to the global clock all processes record their local state at the exact same time

However, in real world distributed systems this protocol cannot actually work. Physical clocks inevitably drifts and network congestion, failures, retransmissions etc... make the message delays unpredictables. So, we need to change the protocol in order to eliminate these 2 assumption.

Removing the Global Clock

IDEA: Instead of asking processes to record their state at a specific time, every process sends a message to p_0 whenever local event happens. Then, p_0 will reconstruct the run to create the global snapshot.



NOTE: At this moment we are still relying on the SYNCHRONOUS NETWORK assumption. Because message delay is bounded by Δt p_0 knows that any event older than Δt has defently arrived.

Removing the Synchronous Assumption

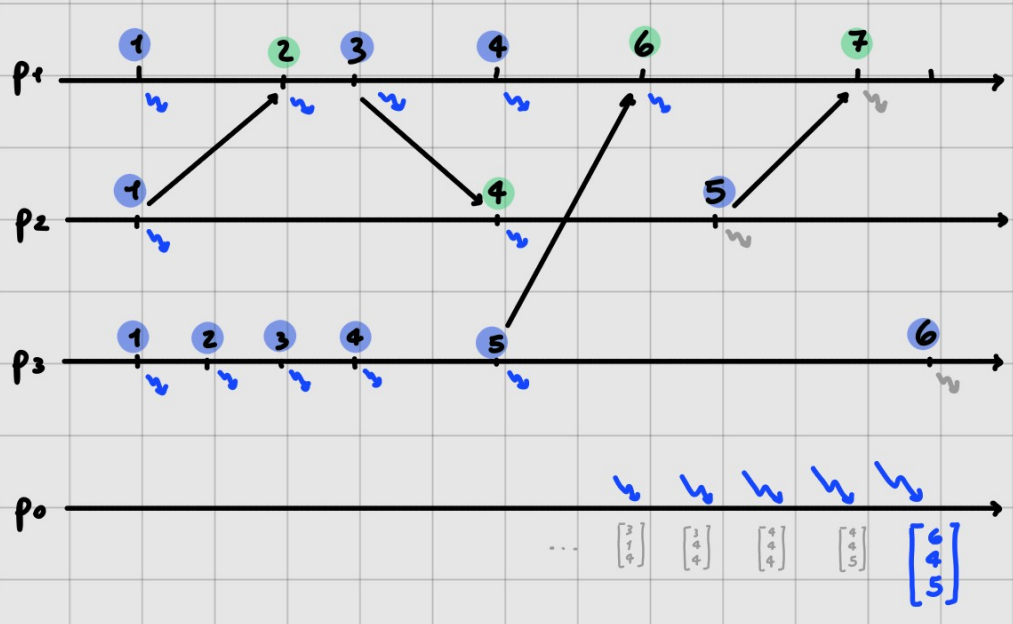
IDEA: To understand the system's global state we do not actually need to know the exact timestamp an event occurred. We only care about the order of the events. So, we can introduce the Clock Condition and the Lamport Clock to reorder messages to their "logical clock".

- (WEAK) CLOCK CONDITION: $e \xrightarrow{\text{happened before}} e' \Rightarrow \underbrace{T(e) < T(e')}_{\text{Timestamp}}$ (NOTE: $\Leftarrow$ it's not true)

- LAMPORTS CLOCK: Each process mantains an integer counter C and attaches it in every message.

  - At each LOCAL event increment $\Rightarrow$ $C = C + 1$
  - Receiving a message with timestamp t $\Rightarrow$ $C = \max(C, t) + 1$

Finally, to reconstruct the order of the events a node must ensure that no message with an older timestamp will arrive. To make this we can use a LAST MESSAGE VECTOR. Each process tracks the latest timestamp received from the others in a vector $[last_1, \dots, last_n]$. Events with timestamps up to $\min(last_1, \dots, last_n)$ can be safely committed because no future message can have smaller timestamp.



PROBLEM: If a process become idle its local counter C stops increasing. Consequently, its entry in the LAST MESSAGE VECTOR remains low freezing the global minimum. In that case the other processes can not commit their events because they are waiting on the silent process.

SOLUTION: Heartbeats / Periodic Null Message

(Strong) Clock Condition

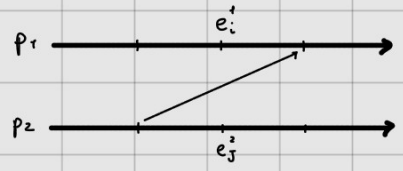
The previous (WEAK) CLOCK CONDITION "$e \longrightarrow e' \Rightarrow T(e) < T(e')$" could not say if e happened before e' or if they were simply concurrent systems. To solve this we want to achieve a STRONG CLOCK CONDITION:

$$e \longrightarrow e' \iff T_S(e) \lesssim T_S(e')$$

To achieve this bidirectional relationship we need to look at the entire history of an event.

Exercise: Is it possible that $e_i^l \not\longrightarrow e_j^m \wedge e_j \not\longrightarrow e_i$?

Yes!



Event's History

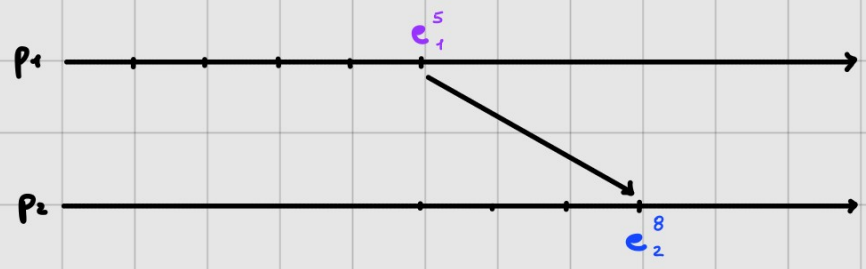
def The History of the event e is the set of events that happened before e
It is formally represented as:

$$H(e) = \{ e' \text{ t.c. } e' \longrightarrow e \}$$

(happened before)

NB if $e_i \longrightarrow e_j$ then $e_i \in H(e_j) \wedge H(e_i) \subseteq H(e_j)$

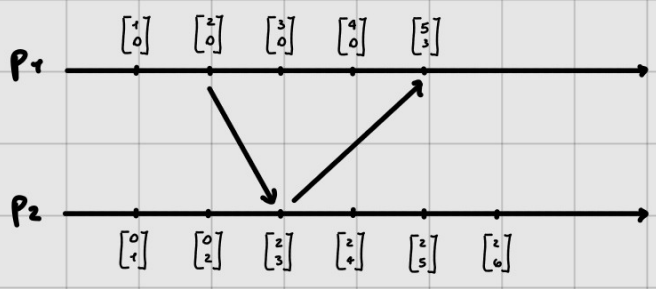
Vector Clock



Now we can define $T_s(e_2^8) = H(e_2^8) = \{e_1^5, e_2^8\}$ and write it "compressed" as a vector (known as Vector Clock) where each element v_i represents the latest know logical clock of the process p_i:

$$T_s(e_2^8) = H(e_2^8) = \begin{bmatrix} 5 \\ 8 \end{bmatrix} \begin{matrix} \leftarrow p_1 \\ \leftarrow p_2 \end{matrix}$$

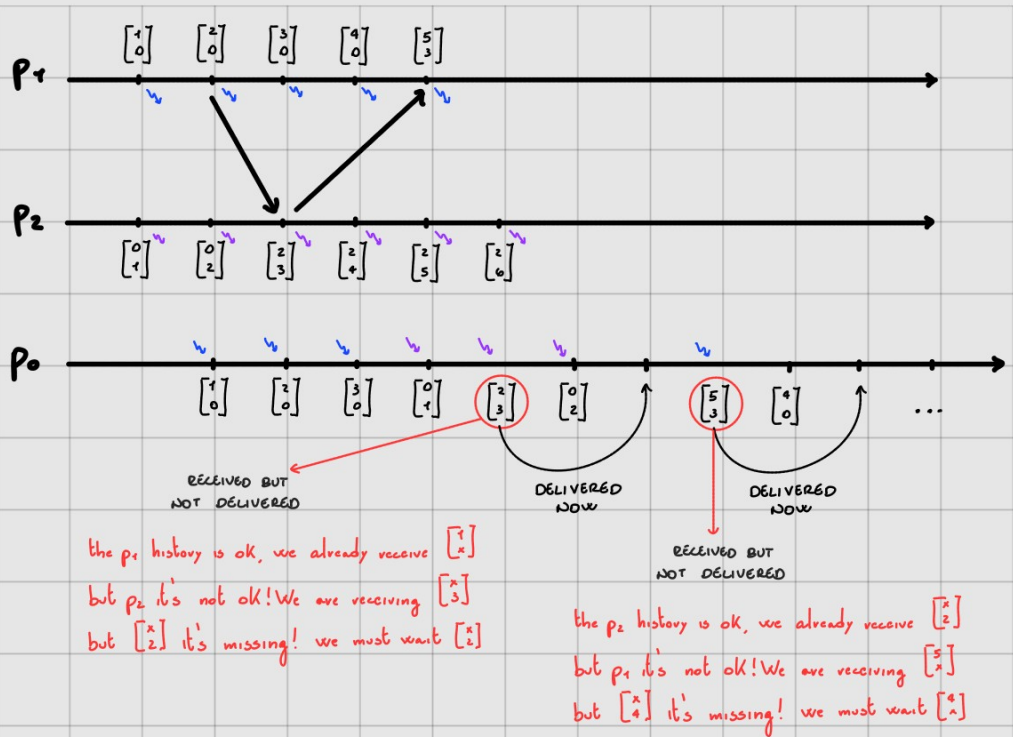
Exercise: Compute the History of each event



At this point we can also define that: "A vector is smaller than or equal to another iff every single component is smaller then or equal to its corrisponding component"
Formally:

$$\begin{bmatrix} x \\ y \end{bmatrix} \leqslant \begin{bmatrix} x' \\ y' \end{bmatrix} \iff x \leqslant x' \wedge y \leqslant y'$$

Because Vector Clock perfectly track all the order dependences, the coordinator p_0 will be able to detect if messages have arrived out of order (and potentially reorder them).



In this way p_0 can "reconstruct the Run" that, thanks to Vecto Clock, it's also CONSISTENT!

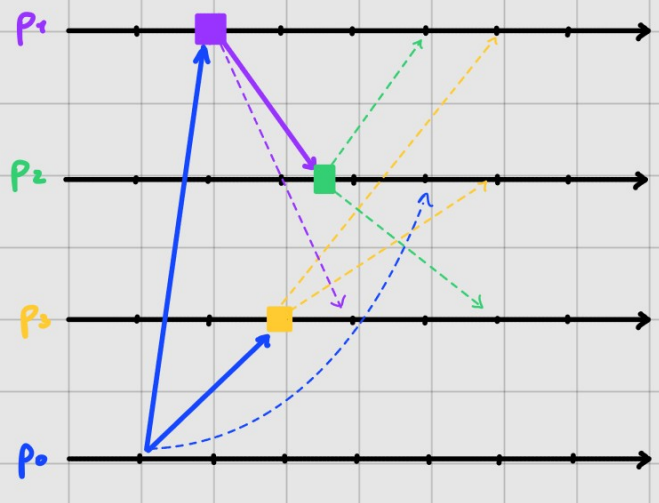
Chandy - Lamport Clock

The Chandy - Lamport Clock allows to take a picture of the global state without halting the underlying execution.

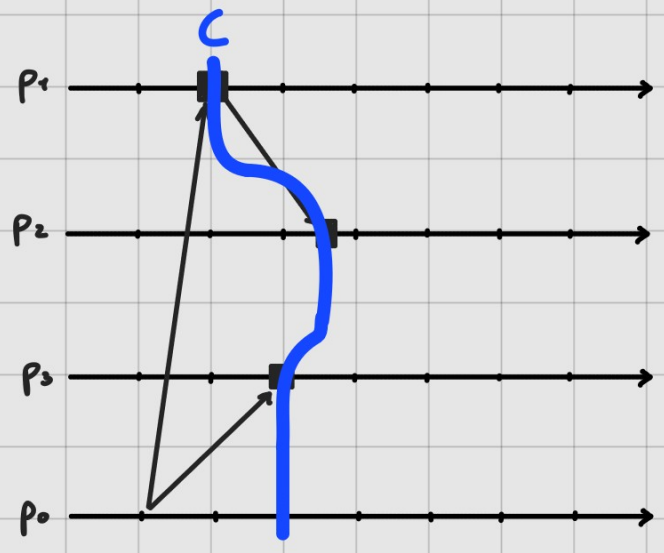
IDEA:

1) Each process sends the "take a snapshot" message to everybody else AS SOON AS it receives the message

   ↳ Example: p_2 sends the message as soon as it is received by p_1, and not from p_0



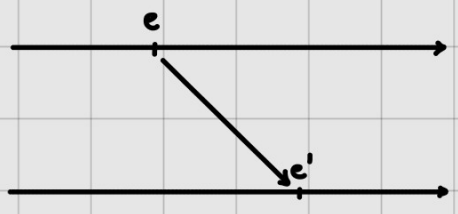
2) The cut on the exact moment each process take the snapshot forms the global state C.



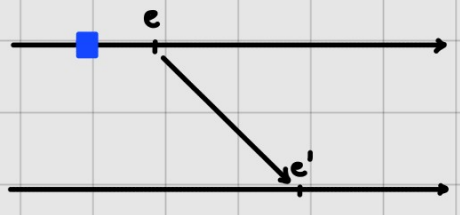
But: "is this cut C consistent?"

Dim. To prove that the Chandy-Lamport algorithm always generates a consistent cut we can use a proof by contradiction

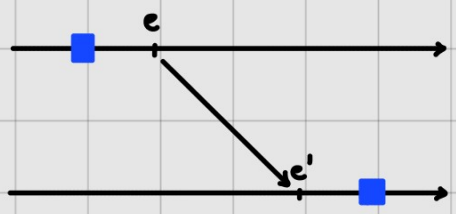
Let's assume that C is the cut genevated from the Chandy-Lamport protocol, and there are 2 events e,e' such that $e \longrightarrow e'$, $e' \in C$ BUT $e \notin C$.



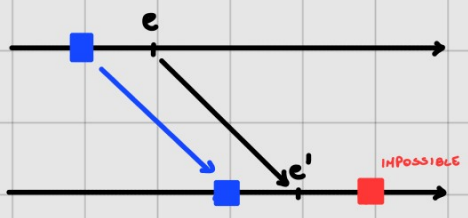
Since $e \notin C$, the "snapshot message" must have arrived before e



And, since $e' \in C$, the "snapshot message" must have arrived after e'

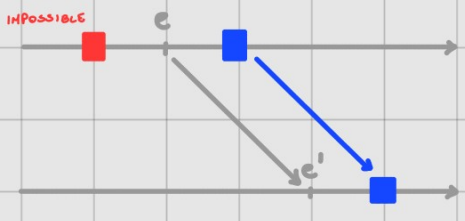


However, since the Chandy-Lamport Clock Protocol works on the assumption of a FIFO CHANNEL, the "snapshot message" send before e MUST arrive before e'



Consequently, both $e, e' \notin C \Rightarrow C$ is consistent $\Rightarrow$ CONTRADICTION (we also have a contradiction on the premise that $e' \in C$)

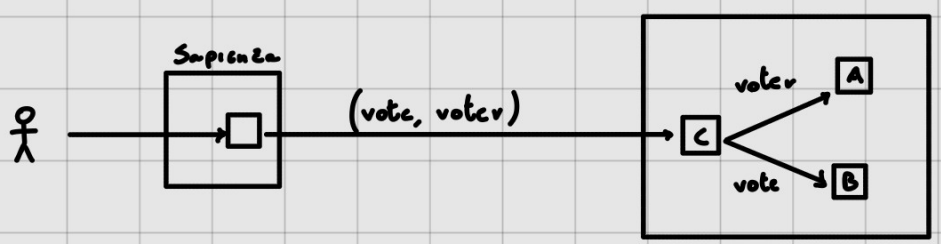
We can also apply the same logic on the "snapshot message" arrived after e'. Since the channel is FIFO it must have been sent after e.



Consequently, both $e, e' \in C \Rightarrow C$ is consistent $\Rightarrow$ CONTRADICTION (we also have a contradiction on the premise that $e \notin C$)

Atomic Commit

Let's start from an example: The 2020 elections in Sapienza. During the COVID pandemic these elections were done remotelly. Consequently they needed a distributed system to collect the (vote, voter) pairs without storing the 2 information together (to ensure the vote anonimity).



(we need to know the voter to avoid that someone could vote twice)

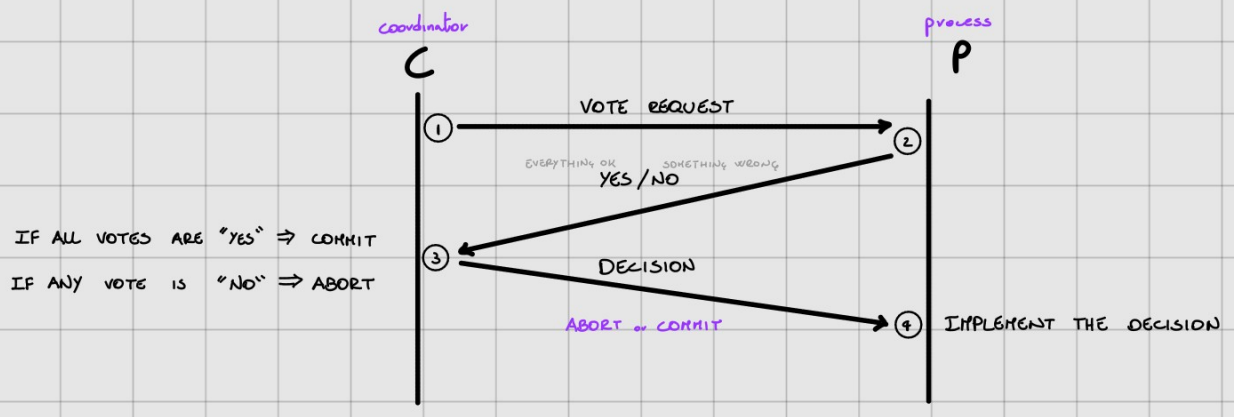
IDEA:

1) register the voter on A
2) register the vote on B
3) delete the pair (vote, voter) on C

all together or none of them

All of these operations SHOULD be done together, as a single ATOMIC COMMIT. However A, B, C could fail during the Transaction... we need a protocol to ensure the atomi commit.

PROTOCOL:



NO UPPER BOUND TO DELIVER MESSAGES

IF WE SEND A MESSAGE INFINITE TIMES, SOONER OR LATER IT WILL ARRIVE

However, we are considering a REALISTIC system (asynchronous and fair-loss). So, to make it work properly we need 4 different ASSUMPTIONS.

ASSUMPTIONS:

SAFETY AC1: All processes that reach a decision must reach the same one (ABORT or COMMIT)

SAFETY AC2: Once a proces reach a decision it doesn't change

SAFETY AC3: COMMIT can be decided only if all processes votes YES

NOTE: if it was an "if and only if" the problem would not be solvable

LIVENESS AC4: if there are NO FAILURES and everybody votes YES, the decision must be COMMIT

FAILURES: But, again, we have to consider failures:

- VOTE REQUEST DOESN'ARRIVE: We can simply set a TIMEOUT. If nothing arrives then Abort.
- VOTE DOESN'ARRIVE: Also the coordinator can set a TIMEOUT. If nothing arrives then Abort.
- DECISION DOESN'T ARRIVE: In this case we cannot set the TIMEOUT... What if we ABORT and then the COMMIT DECISION arrives? All the other processes will commit and our part will be lost invalidating the system. We could proceed in different ways.
    - COOPERATE: thanks to AC1, we can ask to other processes. If anyone aborted we abort; if anyone committed we commit.
    - WAIT FOREVER: Since the channel is fair, sooner or later, the decision will arrive.
    - IMPLEMENT A DISTRIBUTED LOG: So that, in case of crash, we can reload the log and see what happened until now.

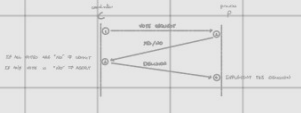
Distributed Log

Unfortunately, we cannot actually create a Distributed Log... It is like an abstraction derived from the collection of the Local Logs of each process. Note that each process (both coordinator or not) must save the local log on its permanent disk, to survive failures. Only by combining the local logs we can reconstruct the whole transaction.

- LOG THEN SEND: Generally, this is the safest approach. If there is a crash the process reads the log, sees that it was supposed to send a message and re-sends it.

- SEND THEN LOG: This is more risky. If there is a crash immediatly after sending a message but before writing the log, after the rebooting the process will have no idea it performed that action.

Let's how this applies to the previous protocol:



- PROCESSES:
  - IF THE VOTE WAS "YES": It's mandatory the Log then Send. Otherwise, with the Send then Log, if the process votes YES, then fails and forget, could resend a different vote ("NO") invalidating the previous message at this point already received by the coordinator

  - IF THE VOTE WAS "NO": Log then Send is better but both are fine. With Send then Log if a process votes "NO" then fails and forget, after the reboot it will abort by default. So the vote doesn't change.

- COORDINATOR: The coordinator crashes, then reboot and read the log:
  1) THERE ARE NO VOTES: probably failed at ①. Re-sends the VOTE REQUEST

  2) THERE IS AT LEAST ONE "NO": Abort directly

3) THERE ARE ALL "YES" BUT SOME VOTE IS MISSING: We could have lost some "no" during the crash (or the vote was "yes" but will never arrive). The coordinato Abort for safety.

4) THERE ARE ALL "YES" BUT NO DECISION LOG: With Log then Send, no decision log mean no decision sent. So we could send COMMIT or ABORT. To stay safe lets assume that we ABORT always in this case. A Send then Log, instead, would be a disaster. Imagine that the coordinator sends "COMMIT" and then crash. After the reboot there is no log, it doesn't know the previous decision and sends "ABORT" for safety. The processes would receive both "COMMIT" and "ABORT"...

5) THERE ARE ALL "YES" AND THE DECISION LOG: In both cases, Log then Send and Send then Log, we simply retransmit the same DECISION as before.

08/10/2026

Limitations

Although the Atomic Commit system works, it has several limitations... First of all, there are the Single Point of Failure and the Unanimity Problem: All processes must vote "YES" to complete the transaction. As a result, if a single process goes down the entire system goes down.

So, we must correct this behaviour. We can do it by introducing the idea of "majority" (rather than unanimity). In this way, if we have 5 processes and 2 of them go down, we can still continue, keeping the system up.

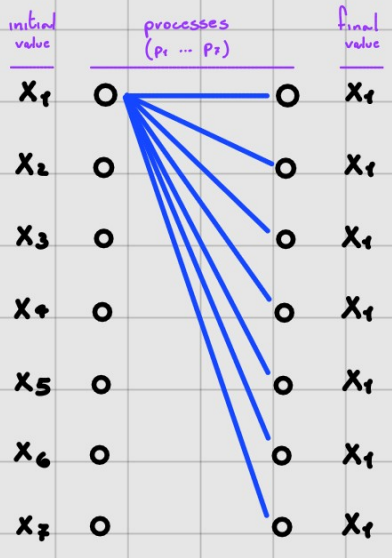
But, as before, we have to manage the CONSENSUS PROBLEM, to guarantee that all processes store the same data (integrity).

The Consensus Problem

Let's assume a distributed system where each process (p_i) has an initial value (x_i). At the end all processes must agree on the same value x, choosen amon $x_1, \dots, x_n$

1) Let's make it easy. Let's assume that the system is SYNCHRONOUS and there are NO FAILURES.

- IDEA: An arbitrary prosses (e.g., p_1) broadcasts its value (x_1) to everybody else. Since there are no failures everybody receives the message and agree on the same value.



2) Let's make it less easy. Lets assume that there could be FAILURES (but the system it's still SYNCHRONOUS).

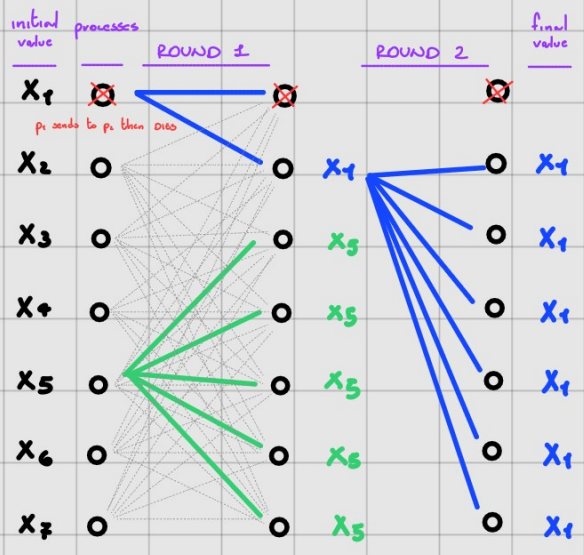
IDEA: Instead of only one process, EACH process broadcasts its value to everybody else. Then, to agree all on a single value, they select only the min value among the received ones.

However, since there are failures we could need to do more ROUNDS to retransmit eventually lost messages.

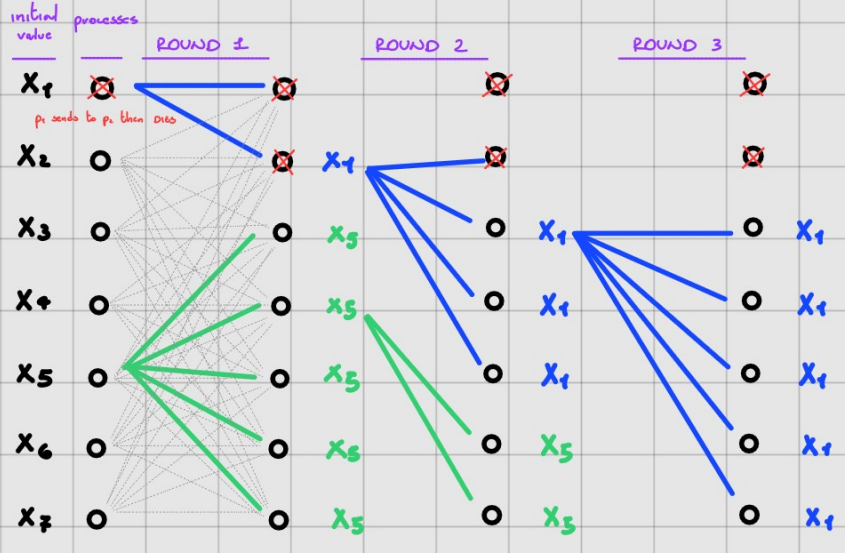
since the system is SYNCHRONOUS we can divide the time in ROUNDS of Δt time

in this case FAIL = CRASH ... so p_1 dies forever after the crash

EXAMPLE: The min value is x_1 BUT p_1 fails after few sends. The second min value is x_5.



So FAILS = 1 ⇒ ROUNDS = 2. What happens with FAILS = 2 ?

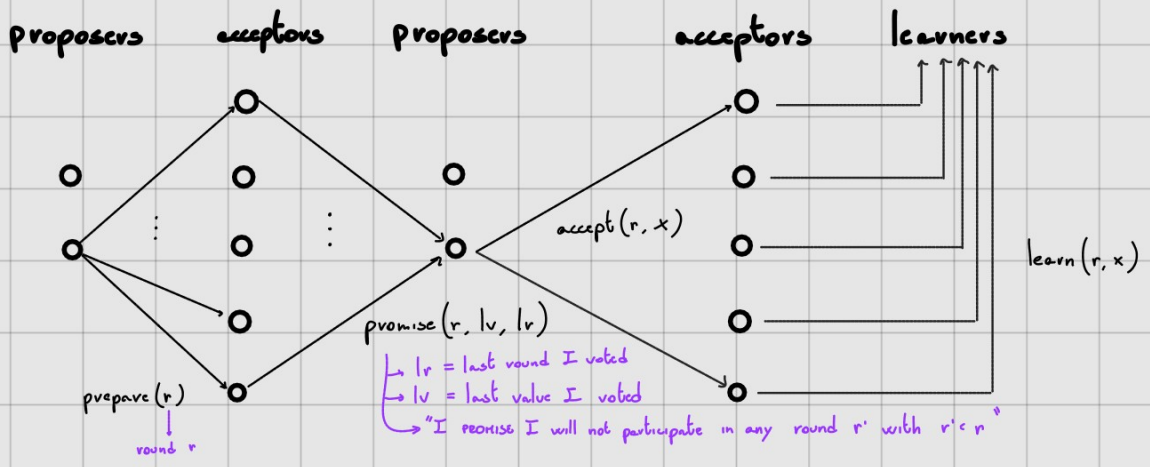


So, in general, we need a minimum of n+1 ROUNDS to solve the problem.

BUT, we are still assuming a SYNCHRONOUS system!
What happens if the system is ASYNCHRONOUS?

This case is much more harder and it needs a new specific well-know protocol.

Paxos



Paxos is a consensus algorithm that operates across 3 different roles: PROPOSER (who suggest values) ACCEPTORS (who actually vote on values) and LEARNERS (who learn/store the choosen value)

ALGORITHM:

1) PREPARE: A proposer initiates a new round by choosing a random number r and sending a promise(r) message to the acceptors

2) PROMISE: When an acceptor receives a prepare(r) message, it replies with a promise(r, lv, lr) message, where:
   - r is the current round
   - lv is the last voted value
   - lr is the last round the acceptor voted in

   In this way the acceptor promises to not partecipate in any round r' < r

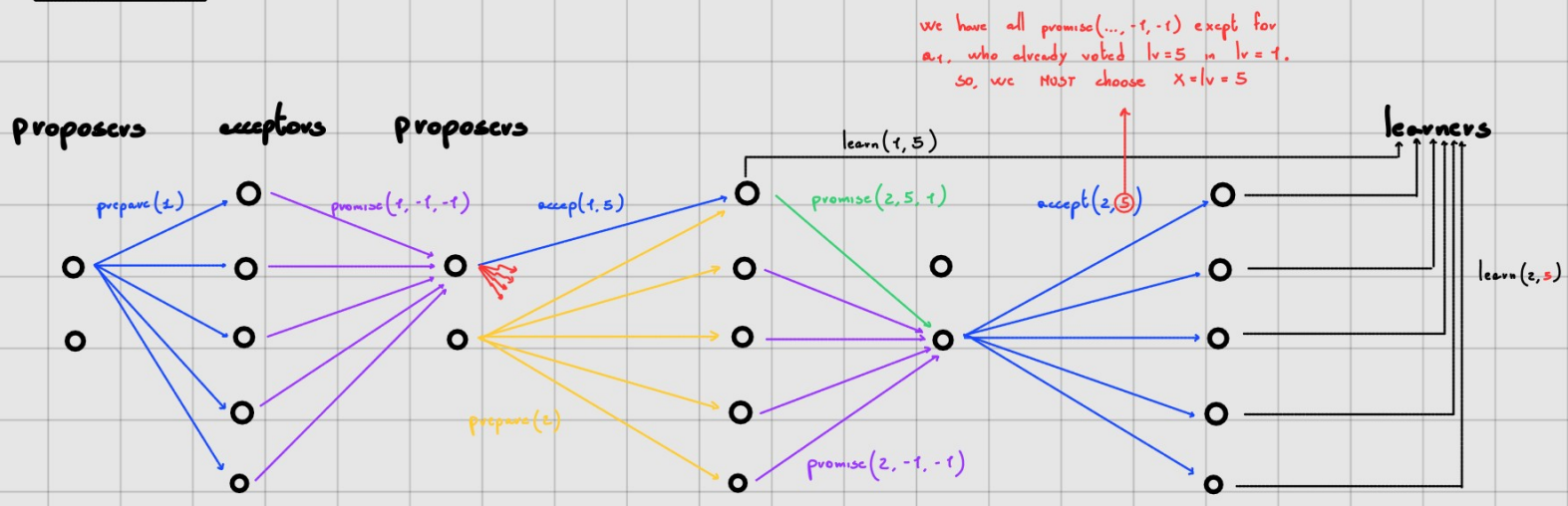
3) ACCEPT: Once the proposer receives promises from a majority of acceptors, it sends an accept(r, x) message to the acceptors, where

$$x = \begin{cases} lv & \text{from the promise with the highest } lr \\ \text{ANY} & \text{if all acceptors have never voted} \end{cases}$$

(lv: LAST VOTED VALUE; lr: THE ROUND lv WAS VOTED IN)

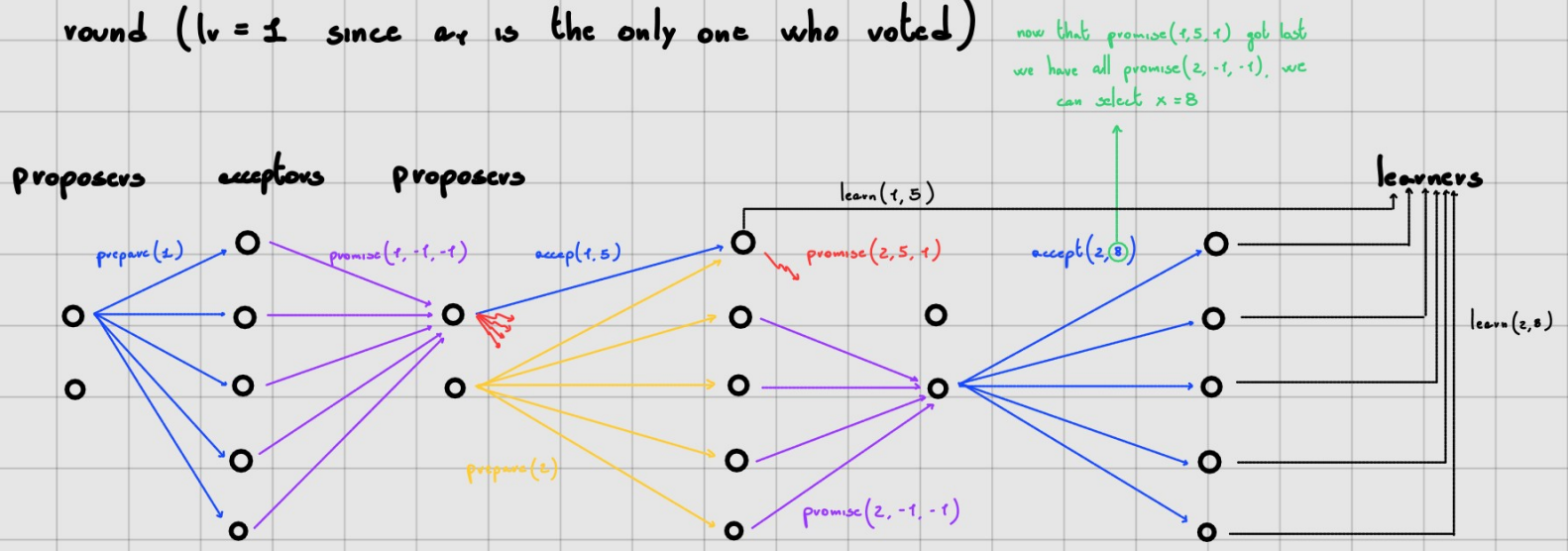
4) LEARN: Once the acceptors receive the accept(r, x) they cast the vote on x sending a learn(r, x) message to the learners, so the system can finalize the decision.

Exercise: a_1 votes for 5 in round 1 and 8 in round 2



WRONG! They all voted 5, not 8.

RIGHT: to ensure that x become equal to 8 the promise(..., 5, ...) MUST be lost! Otherwise, x must be equal to the last voted value (lv = 5) of the max last voted round (lv = 1 since a_1 is the only one who voted)



Consensus Property (CP)

sends a learn(i, x)

def. If acceptor a votes for x in round i, then it is not possible that a different value y was chosen in any of the previous rounds

↳ y gets the majority

DEM: By INDUCTION on i = number of rounds

i = 1 there are no previous rounds ⇒ CP true

i > 1 Assuming that CP is true for all previous rounds $(r_1, \dots, r_{i-1})$. We are currently in round i and we are about to vote x. To reach this voting phase we must have received a promise from a majority quorum (Q). In addition, according to the PAXOS rules, the proposer checks all promises from Q to find the highest last round (lr) any acceptor voted in. Let's call this maximum round J. Clearly, x is the value associated with the round J

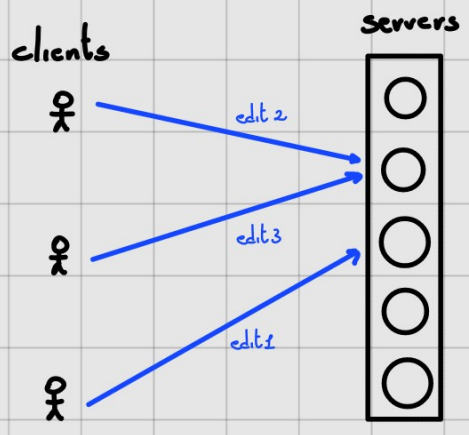
- ROUNDS BETWEEN i AND J: The acceptors in Q did not vote in these rounds. If they had voted the maximum round would not have been J. In addition, any other voted value $y \neq x$ could not have been choosen since the remaining acceptors $(A \setminus Q)$ do not form a majority.

- ROUND J AND BEFORE: We know that at least one acceptor from Q voted x during round J. On the previous rounds we apply the inductive hypoteshis, so it is impossible that any diffevnt value $y \neq x$ was chosen before round J.

BEFORE ROUND J	ROUND J	ROUNDS BETWEEN J AND i	ROUND i
Since someone voted x in round J we can apply the INDUCTIVE HYPOTESHIS	At least one acceptor voted x in this round	the acceptors in Q did not vote in these rounds. If they had voted the maximum round would not have been J The other acceptor $(A \setminus Q)$ do not form any majority. So, they cannot choose any $y \neq x$	we are about to vote x Q = quorum = {acceptors who voted x} J = highest lr from the promises of the acceptors in Q

Multi-Paxos

Let's assume we have 5 servers. Each server is at the same time PROPOSER, ACCEPTOR and LEARNER.

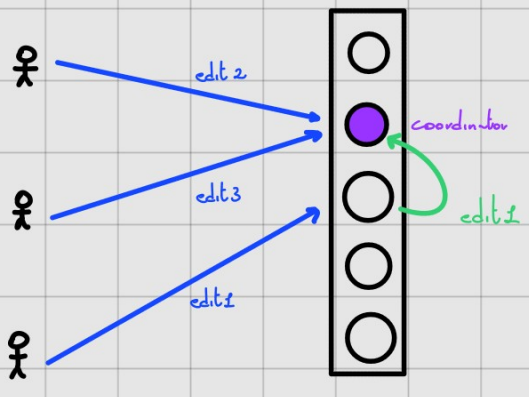


However, a PAXOS instance is useful to agree on a single value (e.g., which edit commit first). In real systems, instead, we have lots of edits. So, we need multiple instances of PAXOS. Consequently, the structure of the messages and the protocol must slightly change to express the number of instance (i):

- prepare (i, n)
- promise (i, n, -1, -1)
- accept (i, n, x)
- learn (i, n, x)

But, at this point, send an individual prepare message for each instance slows down the entire system. We need to optimize it:

1) Coordinator: One of the proposer become the coordinator. Every process sends him the edits. Then, the coordinator will send the prepare message.



2) Optimize the Messages: Instead of sending a prepare message for each instance, the coordinator sends a single prepare(1-∞, r) (instances 1 to ∞, round) cumulative message. It says like "prepare for round r in every instance". In this way, the rounds moves contemporary in all the processes.

Following the same logic, all the acceptors replies with a cumulative message: promise(1-∞, -1, -1) (instances 1 to ∞, round, value x)

The ACCEPT phase, instead, cannot be optimized with a cumulative message. The coordinator should send an accept(i, r, edit) for each instance (i).

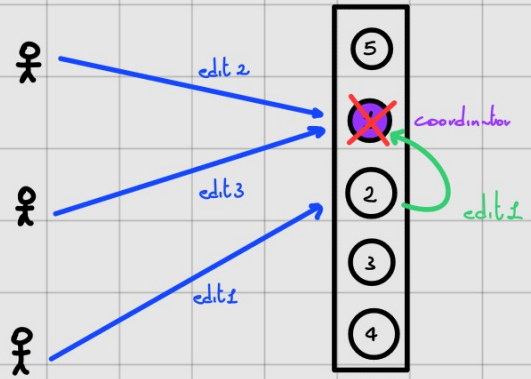
Choose the Coordinator

All the process must agree on the same coordinator. Consequently, we need a protocol to elect the coordinator:

1) Each process sends a heartbeat message ("I'm alive")

2) The process with smaller index become the coordinator (es. server2 receives the heartbeat from server1. So server2 knows himself is not the coordinator)

Clearly, elect a coordinator is a CONSENSUS PROBLEM itself (the same PAXOS resolves). If a heartbeat get lost we could end up with multiple coordinators. Actually, it is not a big problem because it attacks the liveness and not the safety.



If the coordinator p_1 dies he will not send any heartbeat. Consequently, the next immediate index (p_2) will notice the missing message and, after a certain timeout, will became the new coordinator.

Now, p_2 must re-establish authority on the instances. To do it, p_2 sends a new prepare message using a new round, starting from the first instance where there is uncertainty.

- prepare $(4-\infty, 2)$

The acceptors will reply with their promises. Due to the p_1 crash, they can reply in a fragmented way if there are half-finished operations:

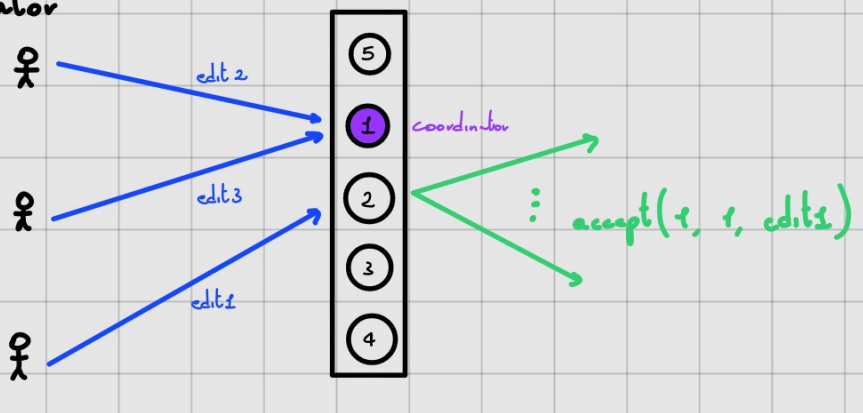
- promise $(4, 2, 1, x)$: indicating that in the instance 4 they already voted for x in round 1.
- promise $(5-\infty, 2, -1, -1)$: indicating that all the instances from 5 to infinite are completely untouched.

Fast-Paxos

Now, let's try to optimize even the accept messages.

- accept $(1-\infty, 1, any)$

A cumulative accept message that allows everyone to accept any value without sending it to the coordinator



Well but, with this configuration the protocol doesn't work. We must change something. Let's assume a configuration similar to the one used for the CP demonstration, with $n = 5$ processes and $|Q| = 3$ quorum.

BEFORE ROUND J	ROUND J	ROUNDS BETWEEN J AND i	ROUND i
Since someone voted x in round J we can apply the INDUCTIVE HYPOTESIS	At least one acceptor in Q voted x in this round (otherwise J would not have been a "last voted round") But, what are the other votes?	the acceptors in Q did not vote in these rounds. If they had voted the maximum round would not have been J The other acceptor $(A \setminus Q)$ do not form any majority. So, they cannot choose any $y \neq x$	we are about to vote x Q = quorum = {acceptors who voted x} J = highest ballot from the promises of the acceptors in Q

Since any

Immagine we are the coordinator and we have to decide to vote x or not based on the other 4 votes. However, since any acceptor can vote any value, we have no information about them. The possibilities are:

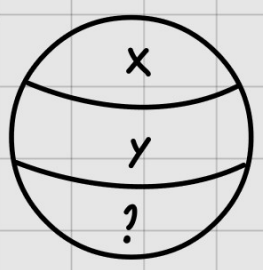
- votes = (x, x, x, ?) : SECURE (last or extremely slow) (x already has the majority. So, we can vote x in round i)
- votes = (x, x, x, y) : SECURE (x already has the majority. So, we can vote x in round i)
- votes = (x, x, y, y) : NOT SECURE (also y could get the majority)
- votes = (x, x, ?, ?) : NOT SECURE (if both ? votes are the same, they could form a majority)

Even one single non-secure case invalidates the whole protocol. To make Fast-Paxos secure we MUST enlarge the quorum size up to (at least) $\frac{2}{3}n$

Dem: If we divide the set of all votes in 3 categories:

- X = {votes for x}
- Y = {votes for $y \neq x$}
- ? = {unknow votes}

We'll need $|X|$ to be at least $\frac{1}{3}$ to ensure that none of the other 2 categories could reach a majority of $\frac{2}{3}$ and safetely vote x.



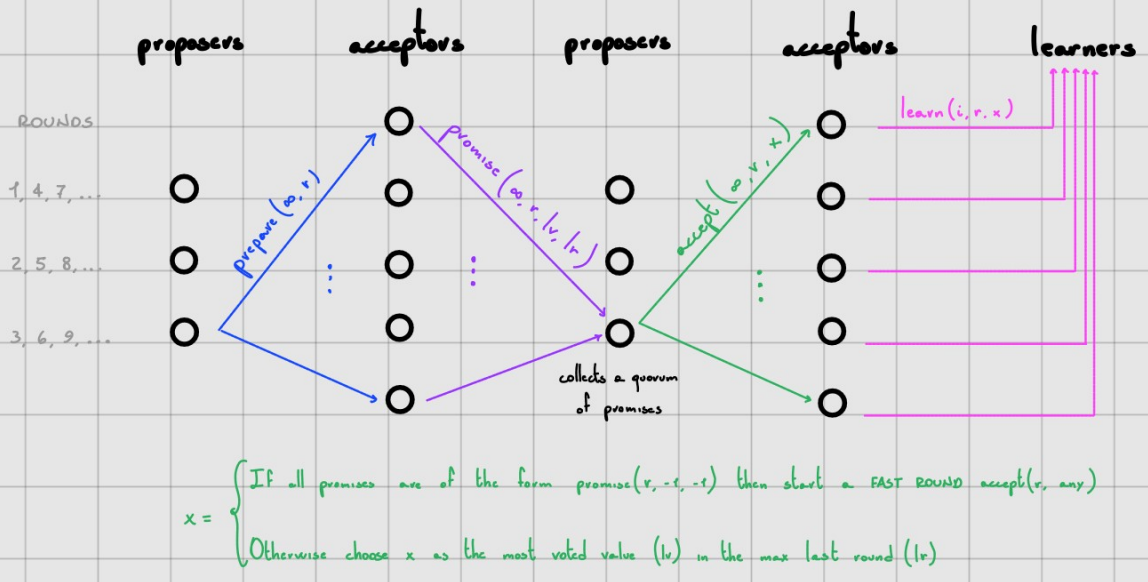
Fault Tolerance

In Classical Paxos we can tolerate a maximum of $f = \lfloor \frac{n-1}{2} \rfloor$ failures, where n = number of acceptors. So, the quorum must be $Q = n - f$

In Fast Paxos, instead, we have:

$$f' = \lfloor \frac{n-1}{3} \rfloor \qquad \text{and} \qquad Q = n - f'$$

Protocol



Protocol:

1) PREPARE: The coordinator starts by sending a cumulative prepare(∞, r) for all future instances

2) PROMISE: The acceptors reply with a cumulative promise(∞, r, lv, lr)

3) ACCEPT: Once the proposer receives promises from a quorum of $\frac{2}{3}n$ acceptors, sends an accept(∞, r, x) to the acceptors, where

$$x = \begin{cases} \text{If all promises are of the form } promise(r, -1, -1) \\ \text{then start a FAST ROUND } accept(r, any) \\ \\ \text{Otherwise } lv \text{ from the promise with the highest } lr \end{cases}$$

4) LEARN: Once the acceptors receive the $accept(i, r, x)$ they cast the vote on x sending a $learn(i, r, x)$ message to the learners, so the system can finalize the decision.

However, in a classical Paxos instance we have 4 messages:

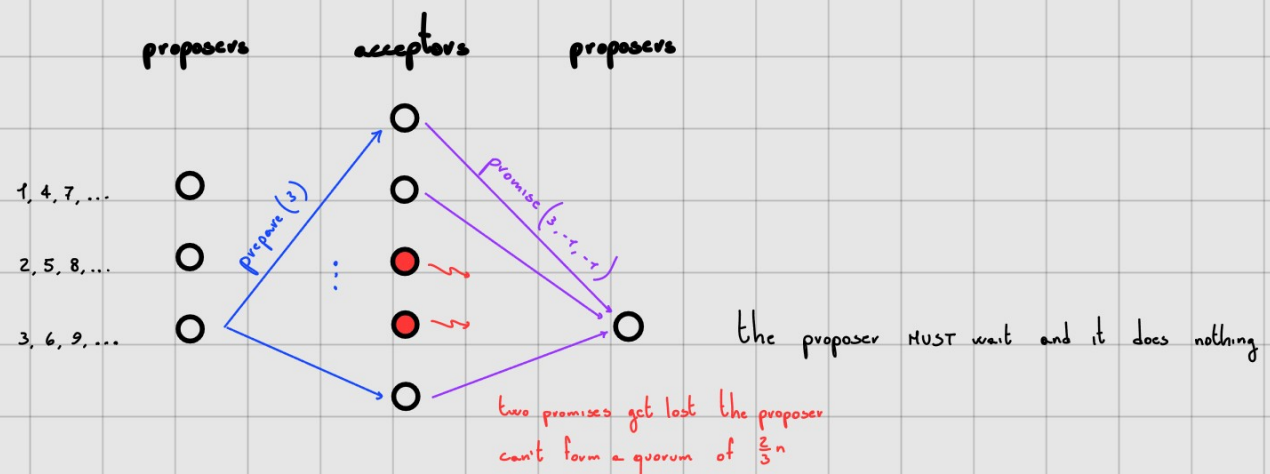
$$prepare(r) \rightarrow promise(r, lv, lr) \rightarrow accept(r, x) \rightarrow learn(r, x)$$

While a Fast Paxos instance requires 5 messages:

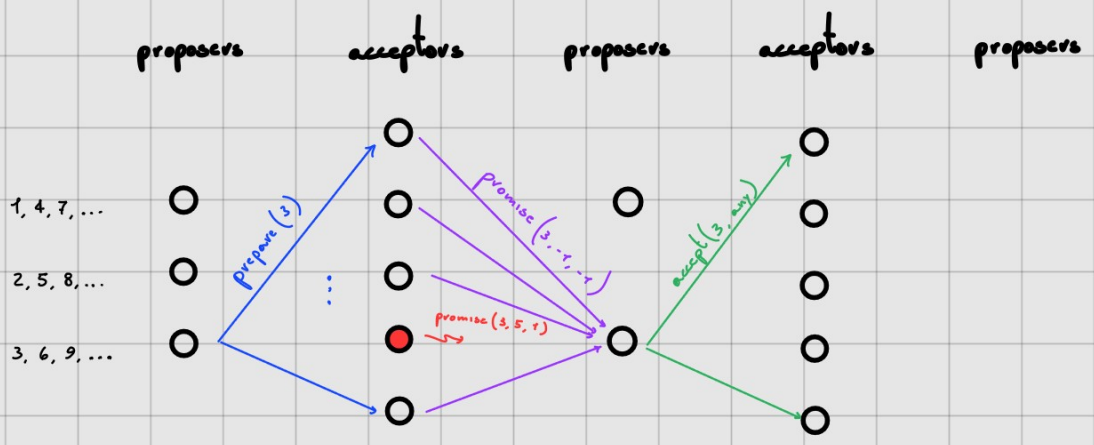
$$prepare(r) \rightarrow promise(r, lv, lr) \rightarrow accept(r, any) \rightarrow \underset{\text{any proposer propose its } x}{accept(r, x)} \rightarrow learn(r, x)$$

So, why do we call it "Fast"? It doesn't seem so... IF WE LOOK AT A SINGLE ROUND! The advantage of Fast Paxos is that the prepare message is just one for all the infinite number of instances, same for the accept any and promises...

Example: Let's see some examples (of a single instance) of Fast Paxos

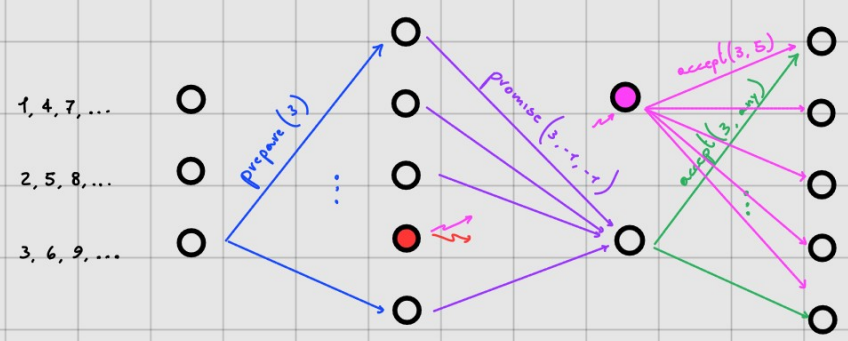


OK, let's now immaging that only one promise get lost (so we can now form a quorum) BUT the lost promise was the only one with value 5.

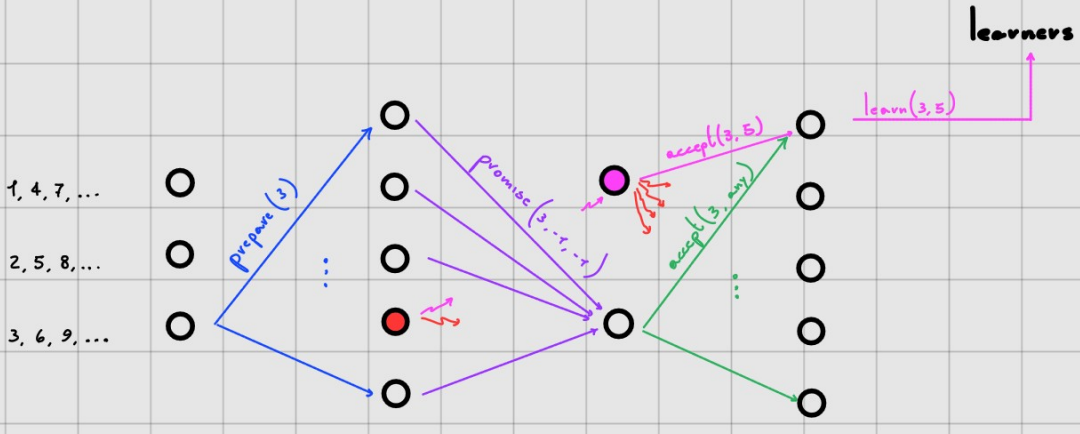


Since there is a value $lv \neq 1$ the proposer must have sent accept(3, 5)... not accept(3, any)... How can we solve this problem?

Since we are in Fast Paxos, when the promise will arrive to any proposer, we'll simply new accept(3, 5) messages!

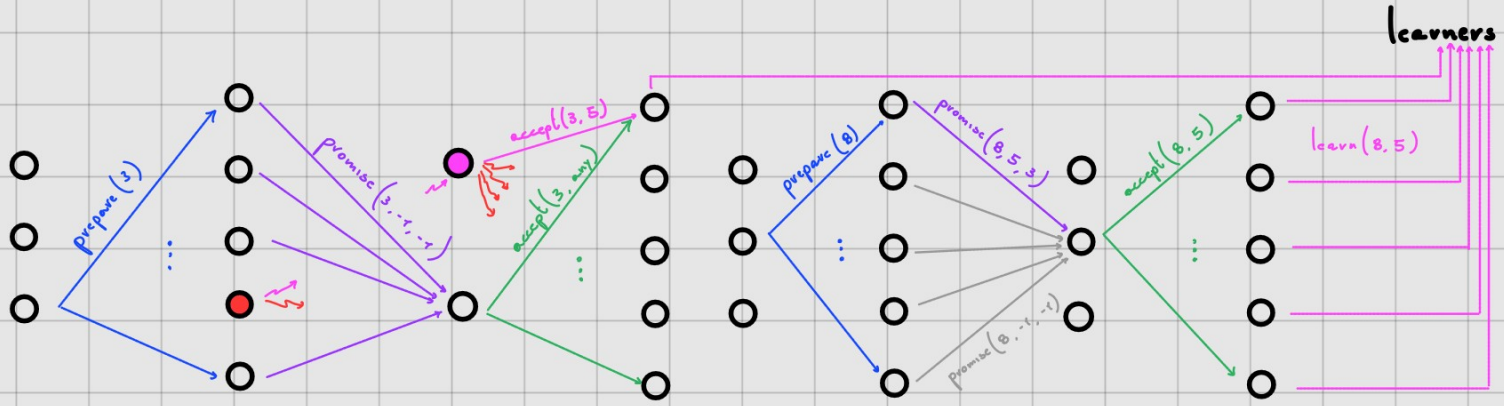


Now, let's even assume that the accept(3, 5) messages get lost all except one

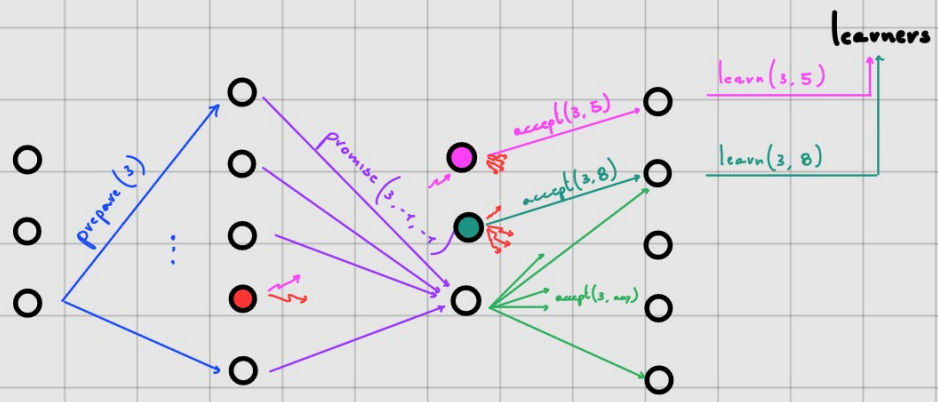


The only acceptor that receives the accept(3, 5) will send the learn, but 5 will not be selected since it does not get the majority. It is possible that in the future 5 will be accepted?

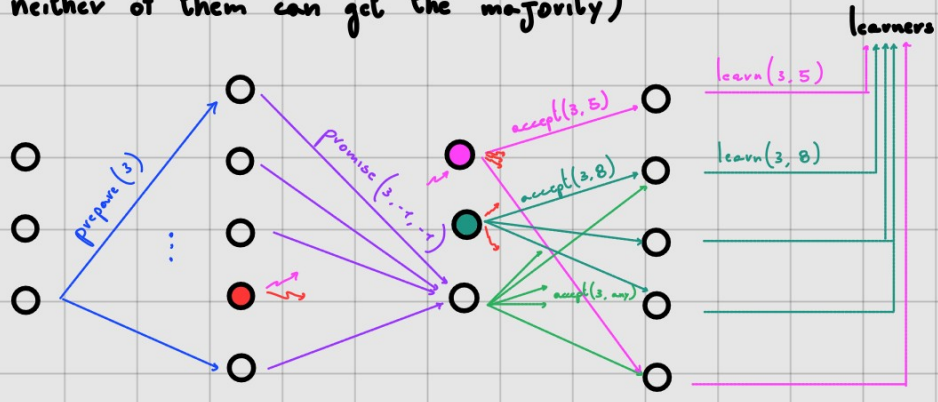
Yes! For example if another proposer (the one in the middle) starts a new round. All the acceptors will reply with promise(r, -1, -1) since they never voted, BUT the one who voted 5 will reply promise(r, 5, 3). So, if the promise arrives, the proposer must choose x = 5 (since it is the lv from the highest lr)



It is also possible that some guys voted for 5 and some for a different value (e.g., 8)



Ov... it is also possible that there are CONFLICTS (e.g., three votes for 8 and two votes for 5, neither of them can get the majority)



To solve the conflict we have to:

1) Wait for a coordinator to notice the lack of decision (e.g., through timeout) and start a new round by sending a prepare message with higher value r.

2) The acceptors will reply with promises containing the conflicting votes from the previous Fast Round

3) The coordinator applies a specific mathematical collision resolution rule to select only one of the conflicting values (e.g., 5 or 8)

4) The coordinator sends a specific accept(i, r, x)

NOT an ACCEP ANY
actually we can't have Fast Rounds after any value gets selected

So, basically, in case of conflict we come back to Classical Paxos.

Conclusion

In conclusion, Fast-Paxos uses all cumulative PREPARE, cumulative PROMISES and cumulative ACCEPT ANY messages to ensure maximum computational speed WITHOUT CONFLICTS...
And this is the problem

- WITHOUT CONFLICTS: The protocol is super fast because the clients write almost immediatly on the acceptors (who accepts any value).

- WITH CONFLICTS: If the clients proposes different values in the same instance, the protocol become slower than the classic version since the coordinator must create a new round and repeat all the protocol.

Raft

Raft is a consensus algorithm that tries to simulate Multi-Paxos to make nodes agree on a SEQUENCE of value, not just a single one.

Differently from Paxos, Raft strongly focuses on the leader election (the coordinator) rather than directly on the value. The system is based on the concept of Replicated State Machine, that can be a database or a document, where each node keeps a LOG of modifications. The goal is to consistently synchronize these logs.

Fault Tolerance

Just like Paxos, Raft assumes there are Crash Failures and NO bizantine nodes (nodes that act arbitrary). The system can tolerate f failures using $n = 2f + 1$ nodes, that's mathematically identical to Paxos' $f = \lfloor \frac{n-1}{2} \rfloor$

Raft, as Paxos, is also SAFETY BUT NOT LIVENESS. On the other hand, a protocol can't be safely, liveness and solve the consensus problem contempovary (according to the FLP paper/theorem that we have not seen)

Leader Election

Time in Raft is divided in Terms, which are periods of time during which a leader holds power. The fundamental election rules are:

1) There can be at most one leader per term. All the other nodes act as followers

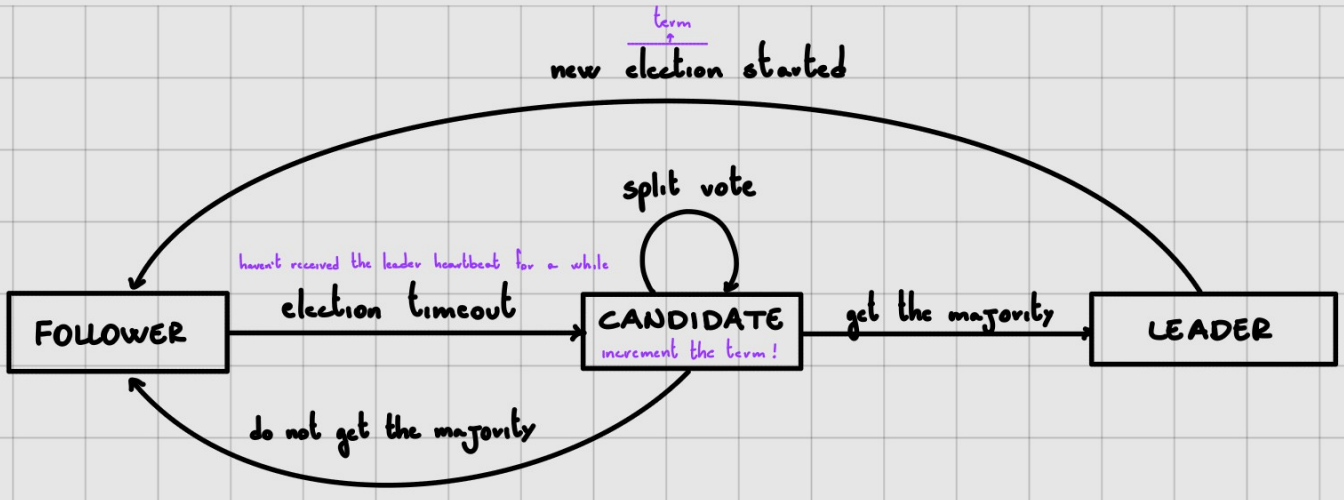
2) Each node can vote for only one candidate per term (and it's valid to vote for yourself)

3) Every leader MUST have all the committed entries

4) Followers can't vote for nodes whose log is smaller

We'll them later

A log is smaller if it contains logs with smaller term OR if contain the same term, it is shorter



Protocol:

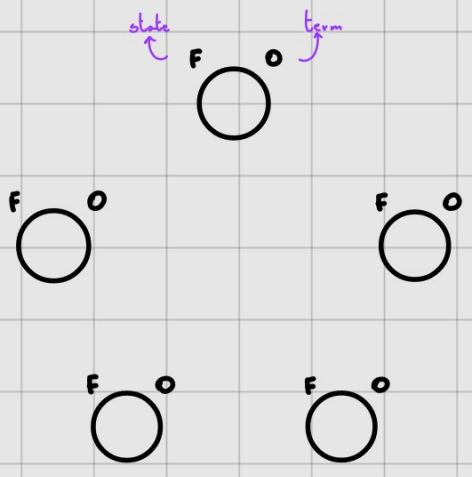
1) FROM FOLLOWER TO CANDIDATE: All nodes start as a Follower. After some time, named election timeout, if the follower haven't received any Leader Heartbeat (it assume the leader crashed and) proposes itself as a candidate (To reduce the probability that two or more nodes propose as candidates contemporary, Raft assigns RANDOM "election timeouts" between 50-300ms to each node for every term).

2) CANDIDATE PHASE: Each candidate increment its local term number, votes for itself and sends a REQUEST VOTE message to all the other nodes (the followers will update their local term number as soon as they receive the vote request)

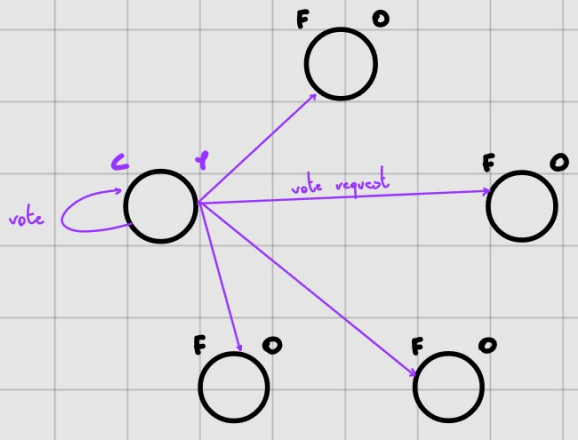
3) VICTORY: If the candidate receives votes from an absolute majority of nodes (50% + 1) it becomes the leader.

4) TIE/SPLIT VOTE: If none of the candidates receives the majority (e.g. 50/50 or 33/33/33) new elections begin (increment the term)

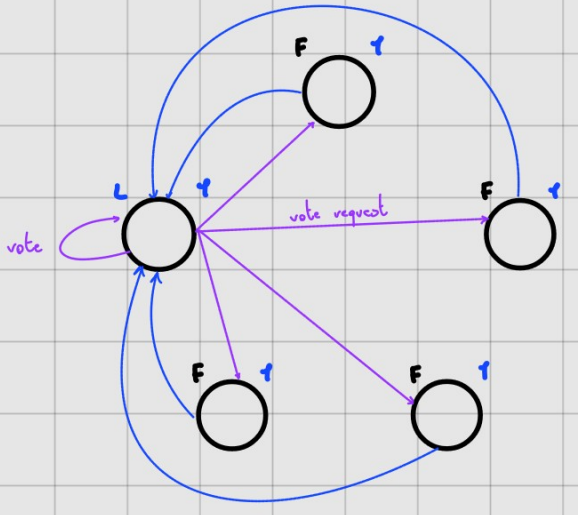
Example: We have 5 nodes. At term 0 they are all Followers



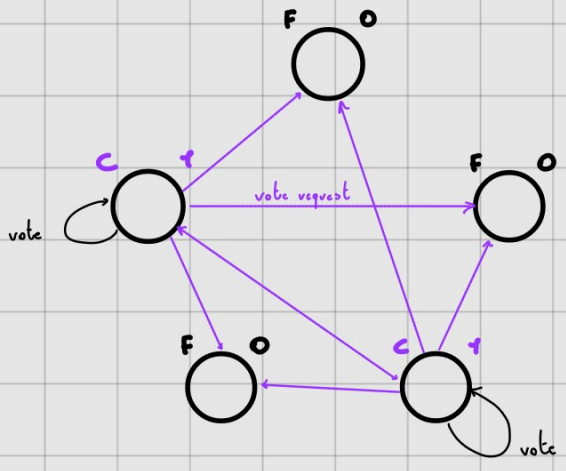
When the first election timeout expires a node propose itself as a candidate, votes for itself and sends the vote requests



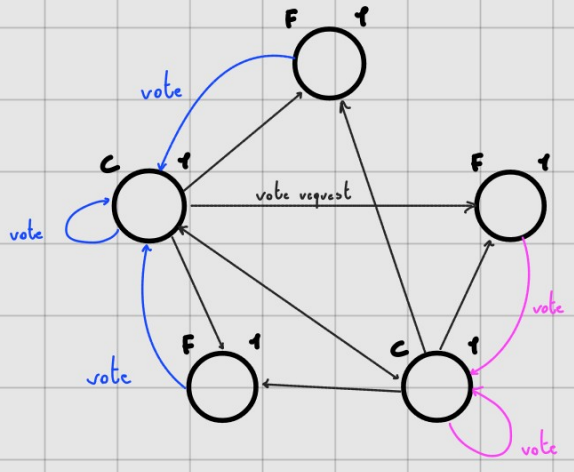
At this point all the followers votes for it, increments their term and it becom the leader for term 1.



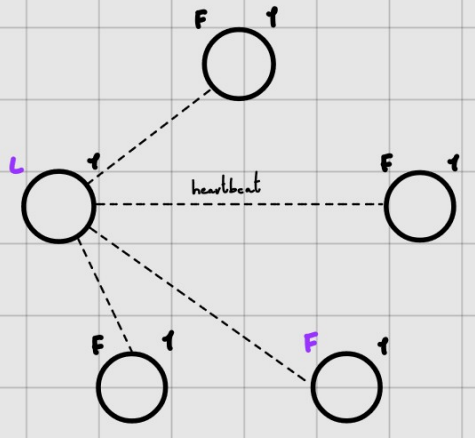
Example 2: Now, let's assume that, at the beginning 2 election timeouts expires at the same time. So, 2 nodes propose as candidates.



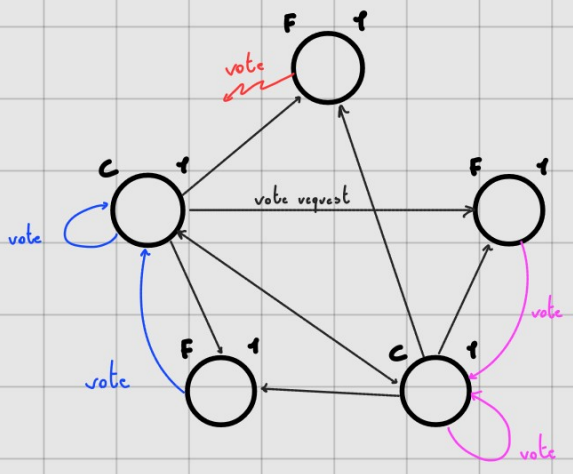
They both broadcasts the vote requests and collects votes.



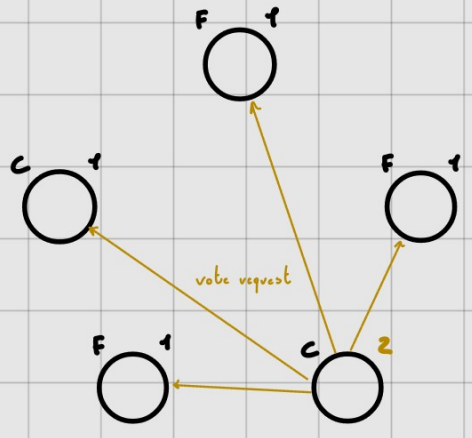
The one that receives the majority will become the leader. Then, the other candidate will receive the leader heartbeat and will say "oh! I've lost the election". So, it will go back to being a follower



What happens, instead, if one of the votes get lost and we have a tie/split vote?



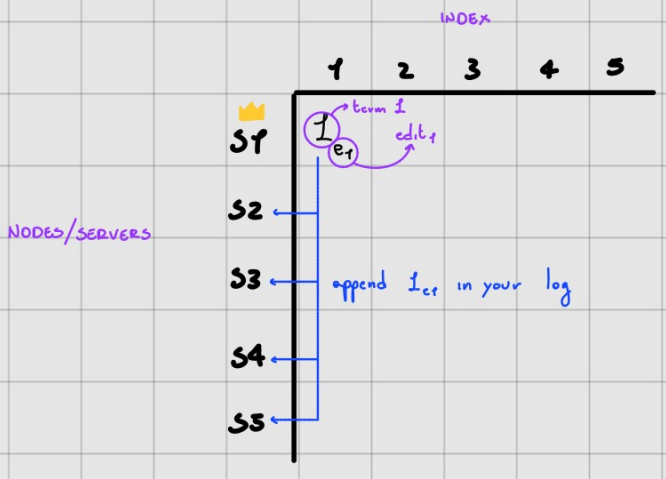
No one will win the elections... So, after a certain timeout one of the two current candidates will start a new term



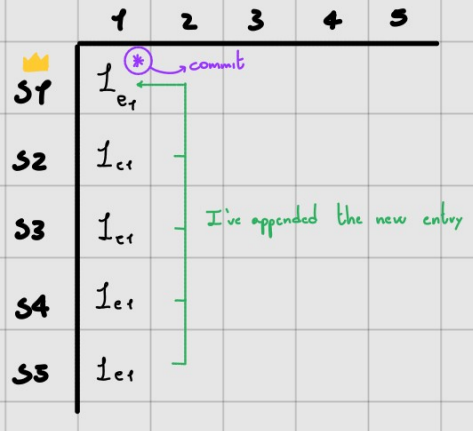
Synchronize the Logs

Once we have a Leader, its main task becomes handling the client requests and safely replicate them in the other nodes logs. In this phase the system behaves like a Distributed Log, where each entry is identified by a numerical index (1, 2, 3, ...)

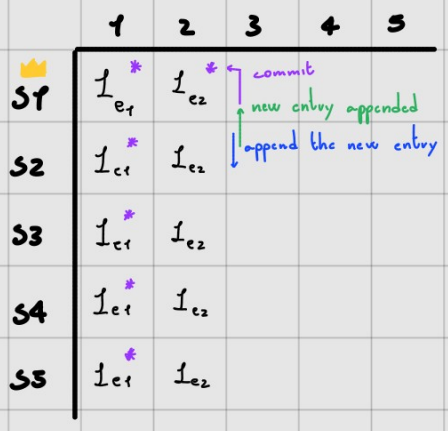
1) Append: When a client sends a new edit, the leader inserts it in its log. Then, the leader sends a message to all the other nodes ordering them to copy this new entry into their respective logs.



2) Leader Commit: Each node, after updating its own log, sends a message to the leader saying "I've appended the new entry". Once the leader has the majority of these messages he can commit the edit.



3) Followers Commit: When the leader receives a second edit (e_2) and sends it to the followers, they will understand that the previous edit (e_1) has been succesfully committed by the leader. So, they commit it too and then stores e_2

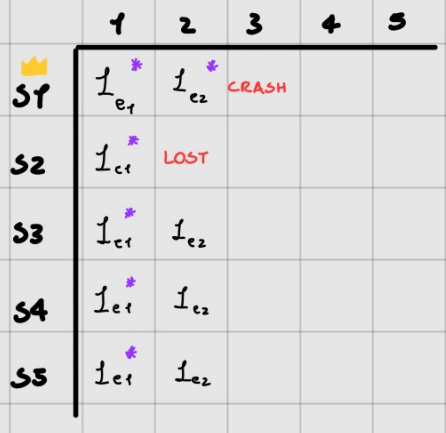


However, during these updates the network could fail, leaving some nodes behind without updates or commits. To prevent that outdated nodes could corrupt the system, Raft enforces two new election rules:

- Every leader MUST have all the committed entries
- Followers can't vote for nodes whose log is smaller

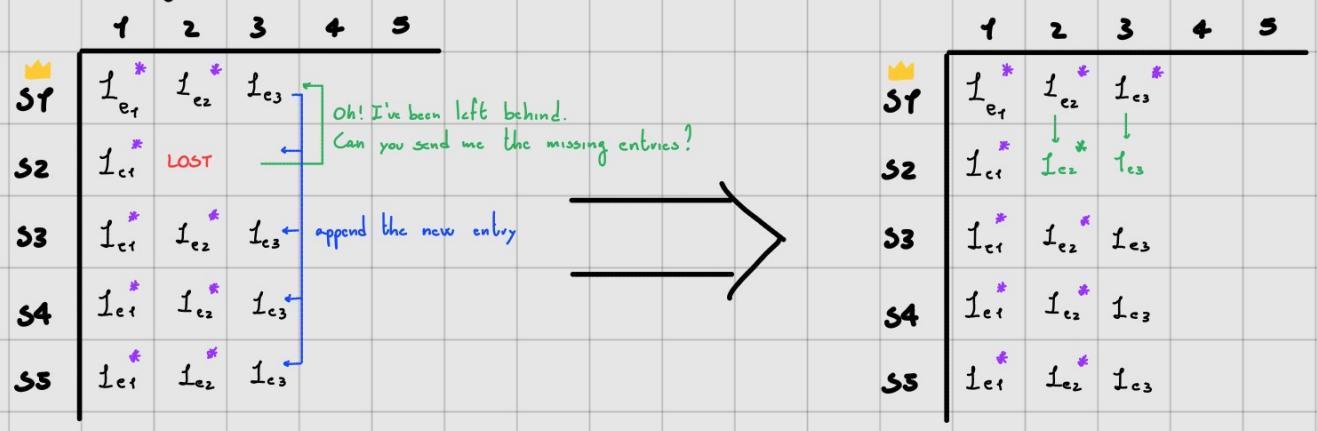
def A log is considered smaller if its last recorded entry has a lower term number than the others. If the final terms are identical, the log that has fewer indicies is simply considered shorter.

Example: In this case S2 can not become a leader



Recovering Lost Messages

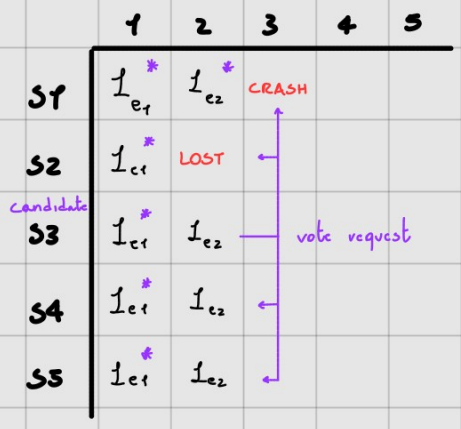
In Raft, when a message is lost, any missing entry will be recovered in background. Simply, at the next edit received by the leader it will send an "append request" to the followers. Any "left behind" node will notice that the edit index doesn't match with its history. So, it will ask to the leader to send him the missing entries so that it can re-synchronize its log.



Leader's Crash

What happens if the Leader crashes?

- A new election begin, so the term increments. According to the election rules only a fully updated node can win. For example, let's assume that S3 proposes itself as a candidate. It will start sending vote request...



- At this point, since the vote request also includes the index and term of the last S3 entry (but not the actual data!), the node S2 immediatly understand that he have been left behind. So, after the elections, when S2 will receive the first heartbeat from the new leader, it will ask him to send all the missing edits to re-synchronize its log.

- Then the protocol will continue as usual, but with a new term due to the new elections